

Pre-Match Predictive Modeling for Five-on-Five Team Games

Executive summary

For pre-match prediction in five-on-five sports and esports, the strongest general-purpose design is **not** a pure Elo system and usually **not** an end-to-end neural network. The most robust architecture is a **hybrid, player-aware, temporally dynamic model**:

dynamic player ratings + lineup/synergy representation + causal recent-form/context features GBDT or hierarchical

This conclusion follows from several strands of evidence. Elo/Bradley-Terry remains an exceptionally strong low-variance baseline and can be interpreted as online logistic learning; recent theory continues to explain why Elo is competitive even under model misspecification and nonstationarity. ¹ Glicko improves the framework by explicitly representing rating uncertainty and time variation, while TrueSkill was designed specifically to infer latent individual skills from **team** outcomes using Gaussian skill distributions and approximate Bayesian inference. ² TrueSkill Through Time goes further by connecting skill states across time and smoothing the whole trajectory rather than estimating each point independently. ³

The major weakness of team-level Elo is structural: a five-person team is not a permanent atomic entity. Players transfer, substitutes appear, roles change, patches or rules change, and particular combinations of players can perform better or worse than the sum of their individual strengths. Research on lineup embeddings, adjusted plus-minus, hypergraph lineup models, and player-context embeddings provides compelling methods for representing those effects. LinNet, for example, modeled NBA lineups as nodes in a performance network and reported 69% out-of-sample matchup accuracy versus 56% for its adjusted-plus-minus baseline; Hypergraph Adjusted Plus-Minus explicitly extends individual effects to interactions and complete lineups. ⁴

For tabular pre-match data, **gradient-boosted trees are the model family I would deploy first**, after a well-designed dynamic rating baseline. XGBoost and LightGBM are mature, high-capacity learners for heterogeneous nonlinear tabular relationships. ⁵ A particularly relevant recent example is PandaSkill: it combines role-specific XGBoost-derived player performance estimates with Bayesian OpenSkill ratings and separate regional/meta ratings; on five years of professional League of Legends data, its pre-match evaluation used rolling one-year training windows followed by one-month tests and obtained roughly 65% overall accuracy, with some cross-region variants around 70% on inter-region games and low reported expected calibration error. ⁶

Deep Sets, GNNs, hypergraphs, and temporal attention models become increasingly attractive as data volume grows. Deep Sets gives the correct inductive bias for an unordered collection of five players,

$$z_{\text{team}} = \rho\left(\sum_{i \in \text{team}} \phi(x_i)\right),$$

while GNNs naturally encode teammate, opponent, role, lineup, and historical interaction graphs. ⁷ GCN-WP provides direct esports evidence: on its professional League of Legends experiment, its best graph-convolutional configuration reported 61.9% accuracy versus 59.7% for a SCOPE/Elo baseline and 57.8% for its random-forest baseline, although the unusual cross-league train/validation/test design and use of accuracy rather than proper probability scores make this a proof of concept rather than a universal benchmark. ⁸

For a forecasting system whose output is a probability, **log loss and Brier score should be primary model-selection metrics; AUC and accuracy should be secondary.** Brier score was introduced specifically for probabilistic forecasts, and proper-scoring-rule theory formalizes why Brier and logarithmic scores reward truthful probability estimates. ⁹ Calibration should be tested explicitly with reliability diagrams and preferably calibration intercept/slope, then repaired from **out-of-time** predictions. Temperature scaling is a strong low-variance default for neural networks; Platt or beta calibration is often preferable for generic scores and tree ensembles; isotonic regression is useful with a large calibration sample; Bayesian Binning into Quantiles is a principled Bayesian alternative. ¹⁰

My ranking for a generic five-on-five pre-match application is therefore:

Rank	Architecture	Overall recommendation	Main reason
1	Dynamic player ratings + GBDT + lineup/ synergy + temporal calibration	Best default	Strong bias–variance tradeoff; transfers skill across roster changes while capturing nonlinear context. Supported conceptually by rating theory, XGBoost/ LightGBM, PandaSkill, and lineup research. ¹¹
2	Hierarchical dynamic Bradley–Terry/logistic model with player and sparse synergy effects	Best statistical/ probabilistic model	Excellent for small-to-medium datasets, uncertainty, cold starts, interpretation, and temporal regularization. ¹²
3	Deep Sets / attention / GNN with dynamic rating priors	Best high-data architecture	Can learn composition and nonlinear interactions without hand-specifying every pair, but needs substantially more data and careful calibration. ¹³
4	Player-level TrueSkill/ OpenSkill/TrueSkill Through Time	Best simple 5v5 rating system	Handles individual players, uncertainty and roster movement much more naturally than team Elo. ¹⁴

Rank	Architecture	Overall recommendation	Main reason
5	Glicko/Glicko-2 applied to team or player aggregates	Strong baseline	Adds uncertainty and dynamics to Elo-like ranking but needs an explicit team-composition layer for full 5v5 modeling. ¹⁵
6	Team-level Elo/Bradley-Terry	Mandatory baseline, rarely final model	Extremely robust, transparent and difficult to beat with weak data, but fundamentally coarse for roster changes and synergy. ¹

The critical engineering lesson is that **the feature timestamp matters at least as much as the algorithm**. A model labeled “pre-match” should have an explicit information cutoff—such as 24 hours before play, announced-lineup time, or post-draft/pre-game time—and every rating, rolling statistic, lineup feature, and calibrator must be reproducible using only information available at that timestamp. Rolling-origin and prequential evaluation are the natural experimental paradigms for this setting. ¹⁶

Modeling objective and probability foundations

A clean formulation starts from a match t , two five-person teams A_t, B_t , information available immediately before the forecast cutoff $\mathcal{I}_t$, and binary result

$$y_t = \begin{cases} 1, & A_t \text{ wins} \\ 0, & B_t \text{ wins.} \end{cases}$$

The objective is not simply to predict the correct winner, but to estimate

$$p_t = P(y_t = 1 \mid \mathcal{I}_t).$$

That distinction is crucial. A classifier that says 0.51 for every winner and 0.49 for every loser can achieve perfect classification while providing almost useless probability estimates; conversely, well-calibrated probabilities allow decisions to respond correctly to the degree of uncertainty. Proper scoring rules such as logarithmic score and Brier score were developed precisely to evaluate probabilistic forecasts rather than only thresholded decisions. ⁹

A particularly useful conceptual decomposition is

$$\text{logit}(p_t) = \underbrace{\Delta R_t}_{\text{long-term strength}} + \underbrace{\beta^\top x_t}_{\text{context}} + \underbrace{F_t}_{\text{recent form}} + \underbrace{S(A_t) - S(B_t)}_{\text{lineup synergy}} + \underbrace{M(A_t, B_t)}_{\text{matchup/style interaction}}.$$

This decomposition is a synthesis of paired-comparison models, dynamic ratings, adjusted-plus-minus ideas and interaction/lineup models. Bradley–Terry/Elo naturally supplies the first term, Glickman-style dynamic models supply uncertainty and temporal evolution, RAPM supplies regularized player contributions, and lineup/hypergraph methods motivate the interaction terms. ¹⁷

The forecast snapshot should be part of the target definition. In esports, “pre-match” can mean before the roster is announced, after the five players are locked, after map veto, or after champion/hero draft. Those are substantially different prediction tasks. A League of Legends study using player–champion experience after champion selection reported 75.1% accuracy before gameplay started, demonstrating how much stronger the information set becomes once composition is known. ¹⁸ Consequently, results from post-draft models should not be compared directly with roster-only forecasts.

For a production system, I would maintain separate forecast products such as:

$$\begin{aligned}p_{\text{early}} &= P(Y \mid \text{known roster/history}), \\p_{\text{lineup}} &= P(Y \mid \text{confirmed five, history}), \\p_{\text{draft}} &= P(Y \mid \text{confirmed five, map/side/draft, history}).\end{aligned}$$

The same principle applies to traditional sports: “morning-of-game” probabilities should not silently use a starting lineup or injury status that was disclosed only minutes before play.

What a five-on-five model must solve

The five-player setting creates four statistical problems that ordinary team Elo largely collapses into one number.

First is **skill decomposition**. The model needs to distinguish team strength from individual player strength so that information follows a player when they transfer. TrueSkill explicitly models each player's latent skill and derives team performance from the members' performances, which is much closer to the actual structure of a five-on-five contest than assigning one scalar to an organization name. ¹⁹

Second is **partial pooling**. A rookie, substitute or newly assembled lineup has little direct evidence. Bayesian ratings and hierarchical models are valuable because estimates can shrink toward role, league or population priors instead of taking a sparse sample literally. Glickman's dynamic paired-comparison work and TrueSkill both explicitly represent uncertainty rather than treating every point estimate as equally reliable. ²⁰

Third is **interaction**. Two individually excellent players may be redundant; a nominally weaker player may create exceptional value with four particular teammates. LinNet, NBA2Vec and hypergraph-adjusted plus-minus all attack versions of this problem by modeling lineups or player context rather than treating player value as entirely additive. ²¹

Fourth is **nonstationarity**. Skills change, teams practice together, players age or improve, rules and game patches change, and competition strength can move across leagues or regions. Dynamic Bradley–Terry, Glicko-style models and TrueSkill Through Time explicitly represent time-varying latent strength. ²²

These considerations strongly favor **player-centered state variables plus a matchup model**, rather than a permanent rating attached solely to a team name.

Rating systems and dynamic skill

The rating-system literature provides the probabilistic backbone for a good pre-match model.

Model	Core mathematical form	Main strengths in 5v5	Main weaknesses in 5v5	Calibration and validation recommendation
Bradley-Terry / Elo	$p(A > B) = \sigma(\theta_A - \theta_B)$. Standard Elo uses $p_A = [1 + 10^{(R_B - R_A)/400}]^{-1}$, then $R'_A = R_A + K(y - p_A)$.	Extremely simple, online, interpretable, data-efficient; strong baseline. Elo is closely related to online optimization of Bradley-Terry/logistic models. ²³	A team scalar cannot identify which player created the rating; roster changes and synergy are awkward; fixed K gives crude uncertainty handling.	Backtest K , home/side advantage and decay by rolling origin. Calibrate the resulting log-odds with Platt/beta scaling if reliability plots show systematic bias.
Glicko	Dynamic paired-comparison model with rating mean and rating deviation; probability is an Elo-like link discounted by opponent uncertainty.	Represents confidence; naturally makes uncertain/inactive competitors less certain; stronger cold-start semantics than Elo. ²⁴	Still fundamentally pairwise; team aggregation must be added externally.	Use chronological rating updates only. Evaluate both mean predictions and calibration by RD/uncertainty strata.
Glicko-2	Extends Glicko with a volatility state controlling how rapidly latent skill is expected to change.	Useful when player strength changes at different rates; uncertainty inflation is attractive around inactivity or role changes. ²⁵	More hyperparameters and assumptions; volatility is not the same thing as lineup uncertainty or synergy.	Estimate volatility settings from historical backtests rather than ranking aesthetics; post-calibrate outcome probabilities if needed.

Model	Core mathematical form	Main strengths in 5v5	Main weaknesses in 5v5	Calibration and validation recommendation
TrueSkill	$s_i \sim N(\mu_i, \sigma_i^2)$, performance $q_i \sim N(s_i, \beta^2)$, team performance $T_A = \sum_{i \in A} q_i$; Bayesian/EP inference updates player posteriors from team outcomes. 19	Designed for multi-player/team contests; uncertainty belongs to individual players; roster changes transfer naturally.	Original model is strongly additive and context-light; specific teammate synergy, map, role and patch effects require extensions.	Convert posterior states into pre-match team probability, then verify reliability temporally. Temperature/Platt adjustment is possible if the performance-variance parameter makes probabilities too sharp/flat.
OpenSkill / Weng-Lin family	Gaussian player-skill distributions with efficient multiplayer/team approximations.	Open practical implementation and flexible multi-team/player support; suitable building block for custom systems. 26	Like basic TrueSkill, context and synergy do not appear automatically.	Same protocol as TrueSkill. Particularly useful as a feature generator for a second-stage classifier.
TrueSkill Through Time	Adds temporal factors connecting $s_{i,t-1}$ and $s_{i,t}$, then performs approximate Bayesian inference over the historical skill path.	Superior conceptual treatment of evolving skill; smooths historical states rather than relying only on forward filtering; software now exists for Python, Julia and R. 3	More computationally expensive and operationally complex; still requires context/synergy extensions for a complete forecast model.	Fit dynamics on past history only for deployment simulation; do not use future smoothing when claiming historical pre-match performance.

Model	Core mathematical form	Main strengths in 5v5	Main weaknesses in 5v5	Calibration and validation recommendation
Dynamic Bradley-Terry / state-space rating	$\theta_{i,t} = \theta_{i,t-1} + \epsilon_{i,t}$, with $P(i > j) = \sigma(\theta_{i,t} - \theta_{j,t})$, or related kernel/time-smoothed variants.	Extremely extensible: player, league, role, map, home and interaction effects can be introduced in one probabilistic model. ²⁷	Inference can become expensive; interaction-rich versions need strong regularization.	Natural candidate for Bayesian posterior-predictive checks plus strict prequential backtesting.
Multidimensional Elo	A cycle-aware parameterization can use $\eta_{ij} = (r_i - r_j) + u_i^\top \Omega u_j$, $\Omega \mathbf{1}^\top = -\Omega$, so matchup-specific non-transitive effects augment scalar strength.	Can represent rock-paper-scissors/style cycles that scalar ratings cannot. Recent work develops this multidimensional Elo family. ²⁸	Higher variance, identifiability issues and much greater data requirements; easy to overfit sparse opponent pairs.	Regularize dimensions heavily and demonstrate improvement on future periods, not merely in-sample pair fitting.

Elo and Bradley-Terry as the mandatory baseline

Bradley-Terry represents latent strength with log-odds:

$$\log \frac{P(A > B)}{1 - P(A > B)} = \theta_A - \theta_B.$$

Elo can then be viewed as a sequential update toward the observed outcome:

$$\theta_{A,t+1} = \theta_{A,t} + \eta_t(y_t - p_t),$$

which is essentially the form of an online gradient step for logistic likelihood. Király and Qian explicitly connect Elo, Bradley-Terry and supervised/online learning; more recent theoretical work analyzes Elo under the Bradley-Terry-Luce model and also under misspecification. ¹

That interpretation suggests an important practical extension:

$$p_t = \sigma(\theta_{A,t} - \theta_{B,t} + \beta^\top x_t).$$

Once written this way, “Elo plus map advantage,” “Elo plus home court,” “Elo plus patch,” and “Elo plus roster continuity” are simply logistic paired-comparison models with covariates. This is generally cleaner than inventing arbitrary Elo-point bonuses. ²⁹

For a five-player team, a first player-level extension is

$$\theta_A = \sum_{i \in A} w_i \theta_i,$$

where the w_i can be equal in a fixed-role esports title or represent expected playing time/possession share in a substitution-heavy sport. TrueSkill's factor-graph construction formalizes essentially this team-from-individuals idea probabilistically rather than merely summing fixed point estimates. ³⁰

Why TrueSkill is particularly relevant to five-on-five games

Under the simplified two-team Gaussian TrueSkill model,

$$s_i \sim \mathcal{N}(\mu_i, \sigma_i^2), \quad q_i \mid s_i \sim \mathcal{N}(s_i, \beta^2),$$

and

$$T_A = \sum_{i \in A} q_i, \quad T_B = \sum_{j \in B} q_j.$$

Ignoring draws, the prior-predictive probability is therefore of the form

$$P(A > B) = \Phi \left[\frac{\sum_{i \in A} \mu_i - \sum_{j \in B} \mu_j}{\sqrt{\sum_{i \in A} \sigma_i^2 + \sum_{j \in B} \sigma_j^2 + 10\beta^2}} \right]$$

for two five-player teams under independent player-performance assumptions. The original TrueSkill paper's factor graph explicitly models individual skill, individual performance, aggregation into team performance, and an observed team ranking. ¹⁹

This gives TrueSkill two properties that are extremely valuable operationally. A player can carry their estimated latent skill to a new team, and a player with few observations contributes greater predictive uncertainty than a veteran. Those properties are difficult to reproduce with a team-only Elo rating without ad hoc roster resets. ¹⁹

The weakness is the additive assumption. If the true strength is

$$T_A = \sum_i s_i + \sum_{i < j} \gamma_{ij} + \gamma_{A, \text{lineup}},$$

basic TrueSkill estimates the first summation but not the interaction terms. The lineup and hypergraph literature provides ways to estimate exactly those omitted components. ³¹

A stronger dynamic statistical model

For a data-rich five-on-five application, I would regard the following as the ideal statistical benchmark:

$$y_t \sim \text{Bernoulli}(\sigma(\eta_t)),$$

$$\eta_t = h_t + \sum_{i \in A_t} w_{it} \theta_{i,t} - \sum_{j \in B_t} w_{jt} \theta_{j,t} + \beta^\top x_t + \sum_{i < j \in A_t} \gamma_{ij} - \sum_{i < j \in B_t} \gamma_{ij}.$$

A useful skill-dynamics prior is

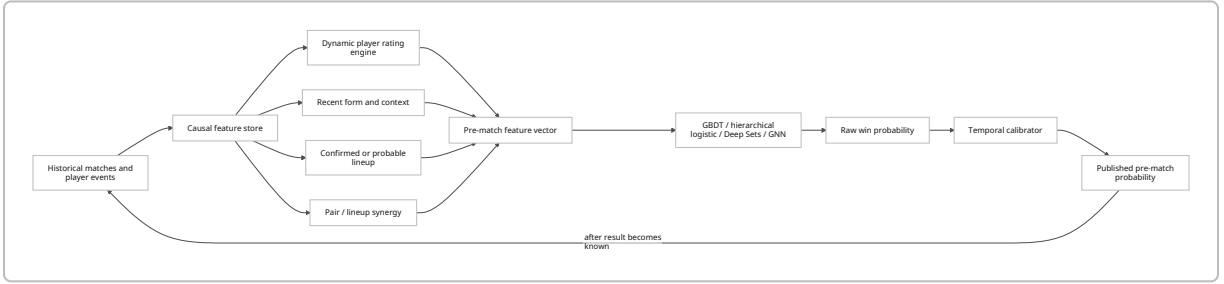
$$\theta_{i,t} = \mu_{r(i)} + \phi_i(\theta_{i,t-1} - \mu_{r(i)}) + \epsilon_{i,t}, \quad \epsilon_{i,t} \sim N(0, q_i \Delta t),$$

where $\mu_{r(i)}$ is a role or league prior and Δt allows uncertainty to grow during inactivity. This is a practical synthesis of dynamic paired-comparison modeling, TrueSkill/Glicko uncertainty, plus-minus player decomposition and regularized synergy. ³²

With sufficiently informative priors, this model can simultaneously solve player transfers, rookies, substitutions, inactivity, contextual advantages, and pair synergy while delivering a posterior distribution rather than merely a point prediction. Its cost is implementation and inference complexity.

Hybrid and pure machine learning

The strongest real-world architecture is usually a **two-layer system**: ratings compress long-term history into low-dimensional state variables, while machine learning explains deviations from what the rating model expects.



The rating component supplies strong regularization and continuity across roster changes; the second-stage model can learn nonlinear context, asymmetry, role interactions, map/side effects and rating uncertainty. PandaSkill is a concrete example of the broader pattern of combining learned player-performance estimates with Bayesian rating updates, while Király and Qian's structured log-odds framework gives the mathematical connection between paired comparisons and richer supervised models.

³³

Gradient-boosted trees on rating and lineup features

For feature vector x , an XGBoost-style model can be written as

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta f_m(x), \quad p = \sigma(F_M(x)),$$

where each f_m is a regularized decision tree and the ensemble is optimized against a classification loss. XGBoost's original KDD work formalized a scalable regularized tree-boosting system, while LightGBM introduced efficiency improvements for large high-dimensional problems. ⁵

For five-on-five forecasting, useful inputs include

$x_t = [\Delta\text{player rating}, \Delta\text{team rating}, \Delta\text{uncertainty}, \text{form}, \text{lineup continuity}, \text{synergy}, \text{map/venue}, \text{side}, \text{patch/rules},$

The major advantages are excellent nonlinear tabular modeling, tolerance of mixed feature scales and missing values, useful feature-attribution tooling, and far lower sample complexity than an end-to-end graph or sequence network. XGBoost was also the model family selected by PandaSkill for its role-specific player-performance component; the authors imposed monotonic constraints for interpretability and reported low ECE for those component models. ³⁴

The weakness is that tree models do not automatically understand that five players form a set, that the same player can appear for different organizations, or that pairwise player relations have persistent meaning. You must encode those structures through ratings, role features, player embeddings, pair features or other representation learning.

Calibration: raw boosted-tree probabilities should always be checked. I would benchmark Platt, beta and isotonic calibration; beta calibration is particularly attractive when the underlying output is already bounded in $[0, 1]$, while isotonic is attractive only when the calibration sample is comfortably large. ³⁵

Evaluation: log loss and Brier first, AUC second, followed by calibration intercept/slope and reliability curves. ⁹

Validation: expanding or sliding walk-forward windows, with hyperparameter selection and calibration themselves performed using historical predictions rather than random pooled folds. Rolling-origin forecasting is well established as an out-of-sample forecasting protocol. ³⁶

Logistic regression, GAMs and hierarchical generalized linear models

A regularized logistic model remains one of the most valuable baselines:

$$p_t = \sigma(\alpha + \beta_R \Delta R_t + \beta^\top x_t).$$

Its biggest advantage is that the output is naturally probabilistic and the marginal effect of every feature can be inspected. Paired-comparison systems themselves fit naturally inside this framework. ²⁹

A hierarchical version can assign partial-pooling priors to players, roles, competitions or regions:

$$\theta_i \sim N(\mu_{\text{role}(i)}, \tau_{\text{role}(i)}^2), \quad \beta_{\text{league}} \sim N(0, \tau_L^2).$$

This is arguably the best choice when you have only a few thousand professional matches and many players, because a highly expressive tree or neural network can otherwise memorize transient identities and lineups.

Calibration tends to be better than for high-capacity classifiers when the logistic specification is reasonably correct, but that should not be assumed. Check calibration slope and intercept and use a Bayesian or Platt correction if necessary. Proper-scoring-rule evaluation remains the same. ³⁷

Deep Sets and permutation-invariant player models

A five-person roster is mathematically a small set. Arbitrarily treating player one, player two, ..., player five as unrelated columns can create unnecessary dependence on ordering. Zaheer et al.'s Deep Sets result motivates functions of the form

$$z_A = \rho \left(\sum_{i \in A} \phi(x_i) \right),$$

which are invariant to permutations of the members. ³⁸

A practical 5v5 matchup network is

$$z_A = \text{Pool}\{\phi(x_i, r_i) : i \in A\}, \quad z_B = \text{Pool}\{\phi(x_j, r_j) : j \in B\},$$
$$p = \sigma(g[z_A, z_B, z_A - z_B, z_A \odot z_B, c_t]).$$

Here r_i can encode a player's role, x_i can contain rating means, rating uncertainty, historical form and embeddings, and c_t contains match context.

This is especially attractive when lineup order is arbitrary. Where roles are structurally meaningful—top/jungle/mid/bot/support in League of Legends, for example—role embeddings should be included rather than forcing complete interchangeability; PandaSkill's separate role models underscore how materially role distributions differ in that title. ³⁹

The main weaknesses are sample efficiency and interpretability. Professional esports datasets often contain only tens of thousands of games rather than millions, so a Deep Sets model should generally ingest pretrained/dynamic rating features rather than be asked to relearn all player skill solely from win/loss labels.

Temperature scaling is the first calibration method I would try for this class of neural model because it adjusts probability sharpness without changing ranking; Guo et al. found temperature scaling surprisingly effective across modern neural networks. ⁴⁰

Graph neural networks and hypergraphs

A graph representation can encode player and team structure explicitly. Let players be nodes and define edges for teammates, opponents, historical co-play, roles or matchup relations. A generic message-passing layer is

$$h_v^{(\ell+1)} = \sigma \left(W_0 h_v^{(\ell)} + \sum_{u \in N(v)} \alpha_{uv} W_1 h_u^{(\ell)} \right).$$

Kipf and Welling's GCN work provides the canonical graph-convolutional basis for learning representations by aggregating neighbor information. ⁴¹

GCN-WP applied graph convolution directly to professional League of Legends. The study experimented with more than 30 match features and compared graph models, random forests and an Elo/SCOPE system. Its best reported GCN configuration achieved 61.9% accuracy in its chosen 2020 cross-league evaluation, against 59.7% for SCOPE/Elo and 57.8% for the best reported random-forest configuration. ⁴²

The architecture is promising, but the result should not be interpreted as “GCNs beat Elo by 2.2 percentage points in general.” The GCN experiment trained on LPL, selected on LCK and tested on LCS, while the SCOPE setup used a different historical rating procedure; furthermore, the paper reports classification accuracy rather than Brier/log loss as its headline comparison. ⁸

Hypergraphs are even more natural for lineups because a five-player lineup is an interaction among more than two nodes. Hypergraph Adjusted Plus-Minus was proposed specifically because standard adjusted plus-minus cannot represent arbitrary higher-order combinations; the method estimates individual, lower-order combination and full-lineup effects. ⁴³

GNN/hypergraph models become my preferred research direction once there are enough observations per player and enough historical co-play to learn interaction representations. Below that regime, regularized pairwise synergy terms or latent-factor interactions are safer.

Temporal neural architectures

Sequence models can represent each player's or team's recent history:

$$h_{i,t} = \text{Encoder}(o_{i,t-k}, \dots, o_{i,t-1}),$$

followed by a Deep Sets or graph layer over the five current members. The conceptual appeal is obvious: it permits nonlinear form trajectories, adaptation after patches, matchup-specific histories and long/short memory.

The difficulty is that a temporal neural model must learn dynamics that Glicko, TrueSkill Through Time or a state-space Bradley–Terry model already encode with strong statistical structure. TrueSkill Through Time explicitly connects skills across time rather than treating observations as exchangeable. ³ For modest professional datasets, I would therefore feed a neural model **dynamic rating states and recent summaries**, rather than replace them completely.

Roster, lineup, temporal, and synergy engineering

In five-on-five forecasting, feature engineering should be organized around **latent ability, composition, context and time**, not around indiscriminately accumulating hundreds of historical averages.

Feature family	Example mathematical construction	Recommended interpretation
Long-term player skill	$\sum_{i \in A} \mu_{i,t} - \sum_{j \in B} \mu_{j,t}$	Player-level TrueSkill/OpenSkill/Glicko/dynamic-BT state; follows players across organizations. ⁴⁴

Feature family	Example mathematical construction	Recommended interpretation
Skill uncertainty	$\sum_{i \in A} \sigma_{i,t}^2 + \sum_{j \in B} \sigma_{j,t}^2$	Rookies, substitutes and inactive players should make the forecast less certain, not merely lower rated. ²⁰
Opponent-adjusted recent form	$\bar{z}_t = \frac{\sum_{k < t} e^{-(t-k)/\tau} (z_k - E[z_k \text{opp}])}{\sum e^{-(t-k)/\tau}}$	Prefer residual performance after accounting for opposition to raw last- N win rate. Dynamic rating research motivates time-sensitive strength. ⁴⁵
Roster continuity	games/days together; pair and trio co-play counts; fraction of recent games using today's five	Detect newly assembled lineups independently of individual skill. Lineup-level research shows composition itself carries information. ³¹
Pair synergy	$S_A = \sum_{i < j \in A} \gamma_{ij}$	Regularized residual value of two players appearing together after controlling for player main effects. ⁴⁶
Latent synergy	$\gamma_{ij} = u_i^\top u_j$	Low-rank alternative when explicit player-pair parameters are too sparse; player-context embeddings such as NBA2Vec motivate this representation-learning view. ⁴⁷
Full-lineup effect	$\gamma_{\{i_1, \dots, i_5\}}$ with strong shrinkage	Useful only with enough repeated lineup exposure; hypergraph models explicitly target this higher-order structure. ⁴³
Competition strength	player regional state + league/region meta-state	Essential where competitive pools rarely meet. PandaSkill's contextual/meta system materially improved cross-region prediction. ⁴⁸
Context	map, side, venue, home, surface, tournament stage, best-of format, patch/rules	Include only variables genuinely known at the declared forecast cutoff.
Composition/draft	champion/hero/player-role embeddings and interaction terms	Extremely informative when forecasting occurs after draft; the 75.1% player-champion experience result illustrates how different this target is from pre-draft prediction. ¹⁸

Player value should be opponent-adjusted

Raw player box-score averages are among the easiest features to misuse. High-resource players often have high statistics partly because their teams are already strong, while support roles may generate value through variables that do not resemble scoring metrics.

Regularized adjusted plus-minus gives a general solution. In its simplest form, for stint or possession-level response D ,

$$D = X\beta + \epsilon,$$

where $X_{si} = +1$ if player i is on one side of stint s , -1 if on the opponent and 0 otherwise. Ridge regularization yields

$$\hat{\beta} = \arg \min_{\beta} [(D - X\beta)^\top (D - X\beta) + \lambda \|\beta\|_2^2],$$

equivalently

$$\hat{\beta} = (X^\top X + \lambda I)^{-1} X^\top D.$$

The regularization is essential because teammates frequently appear together and their contributions are consequently highly collinear. ⁴⁹

For esports, the same principle can be used even without possession-level stints: model historical player performance conditional on teammates, opponents, roles and context, then use the residual or player coefficient as a causal-ish strength feature. PandaSkill uses a different but related strategy: role-specific XGBoost models transform previous end-game individual statistics into player-performance scores and then feed those scores into OpenSkill updates. ³⁴

Importantly, those post-game statistics may update a player's rating **after that game** and become usable for subsequent matches; they cannot be used directly as features for predicting the game from which they came. PandaSkill's separate pre-match forecasting experiment uses the skill ratings that existed before each game and evaluates them with a rolling historical window. ⁵⁰

Lineup and substitution handling

When the exact five-player lineup is known,

$$R_A = \sum_{i \in L_A} \mu_i + S(L_A)$$

is preferable to an organization-level rating.

When the lineup is uncertain, do not silently substitute “probable starters” and pretend there is no uncertainty. Marginalize:

$$P(A > B \mid \mathcal{I}) = \sum_{L_A, L_B} P(L_A \mid \mathcal{I}) P(L_B \mid \mathcal{I}) P(A > B \mid L_A, L_B, \mathcal{I}).$$

This propagates roster uncertainty into the match probability. It is especially useful before official roster announcements.

In substitution-heavy sports, a game's effective team strength can instead be modeled through projected lineup exposure. Let π_ℓ be the expected fraction of possessions or minutes played by lineup ℓ ; lineup embeddings or RAPM/hypergraph estimates can then supply θ_ℓ . The sparse-lineup problem is exactly why regularization and lower-order player/pair components are critical. LinNet and hypergraph APM are particularly relevant references here. ³¹

In esports, the analogous uncertainty often occurs between games/maps rather than through continuous substitution: a best-of series can have different five-player rosters, maps, sides or drafts. A game-level model should therefore generate p_g separately for every game from the information available before that game rather than assuming that a single team probability applies unchanged to an entire series.

Synergy should be residual, not raw co-win rate

A common but flawed synergy feature is

win rate of player pair (i, j) .

That statistic confounds player quality, teammate quality, opposition and schedule. A better model is

$$\eta_t = \sum_i x_{it}\beta_i + \sum_{i < j} x_{it}x_{jt}\gamma_{ij} + c_t,$$

with strong shrinkage on γ_{ij} .

A simple empirical-Bayes approximation is

$$\tilde{\gamma}_{ij} = \frac{n_{ij}}{n_{ij} + \kappa} \hat{\gamma}_{ij},$$

so pairs with little shared history shrink toward zero. HAPM generalizes the idea beyond pair interactions to larger subsets and entire lineups, while LinNet learns lineup-level embeddings from a performance network. ⁵¹

NBA2Vec offers another attractive direction: its player embeddings are explicitly contextualized by teammates and opponents and were trained from millions of play-level observations, demonstrating that representation learning can recover player roles/styles from interaction context. ⁴⁷

For very sparse esports data, I would use a factorized interaction model,

$$\gamma_{ij} = \mathbf{u}_i^\top \mathbf{v}_j,$$

rather than a free parameter for every pair. With more data, a GNN or hypergraph can learn these relations jointly.

Temporal dynamics and regime shifts

The easiest temporal feature is an exponential average,

$$m_t = \frac{\sum_{k < t} \exp[-(t - t_k)/\tau] z_k}{\sum_{k < t} \exp[-(t - t_k)/\tau]}.$$

The half-life τ should be tuned by walk-forward backtesting rather than fixed arbitrarily.

A state-space rating is stronger:

$$\theta_{i,t} = \theta_{i,t-1} + \epsilon_{i,t}, \quad \epsilon_{i,t} \sim N(0, q\Delta t).$$

Long inactivity then increases posterior uncertainty, echoing the motivation of dynamic paired-comparison/Glicko systems. ¹⁵

Mean reversion can be introduced with

$$\theta_{i,t} = \mu + \phi(\theta_{i,t-1} - \mu) + \epsilon_{i,t}, \quad 0 < \phi < 1.$$

For games with major patches, seasons or rule changes, I would allow either a regime-specific intercept/embedding or an increase in process variance around the regime boundary. PandaSkill explicitly cites meta shifts as a reason for using recent rolling windows in its pre-match experiment, while TrueSkill Through Time provides a general framework for time-coupled skill evolution. ⁵²

A player changing region/team should likewise trigger **more uncertainty**, not necessarily a complete deletion of prior skill. PandaSkill's contextual/meta model explicitly resets contextual uncertainty when a player's competitive context changes while retaining the broader rating structure. ⁵³

Calibration, evaluation, and validation

A probabilistic pre-match model should be treated as a **forecasting system**, not merely a classification system.

Calibration methods

For raw model score s , **Platt scaling** fits

$$p_{\text{cal}} = \sigma(as + b).$$

It is a strong low-variance default when raw logits or scores are approximately correct up to a scale and offset. Calibration studies by Zadrozny and Elkan and by Niculescu-Mizil and Caruana established post-hoc calibration as an important component of probabilistic classification. ⁵⁴

For an uncalibrated neural logit z , **temperature scaling** uses

$$p_{\text{cal}} = \sigma(z/T)$$

for binary prediction, or $\text{softmax}(z/T)$ for multiclass prediction. It changes confidence without changing the class ordering; Guo et al. found this simple one-parameter method surprisingly effective on modern neural networks. ⁴⁰

Isotonic regression fits a monotone calibration function

$$g^* = \arg \min_{g \text{ nondecreasing}} \sum_i (y_i - g(s_i))^2.$$

Its advantage is flexibility without assuming a sigmoid shape; its weakness is variance. With a small calibration period, the resulting step function can overfit, so I prefer Platt/beta/temperature scaling in low-data professional leagues and isotonic only when the historical calibration sample is substantial. Isotonic probability calibration is part of the established classifier-calibration literature. ⁵⁴

Beta calibration operates naturally on an existing probability p :

$$p_{\text{cal}} = \sigma[a \log p - b \log(1 - p) + c].$$

Kull, Silva Filho and Flach introduced it as a richer alternative to logistic calibration for binary classifiers. Its important practical advantage over Platt scaling is its ability to represent asymmetric distortions in an already probabilistic output. ⁵⁵

For **Bayesian calibration**, Bayesian Binning into Quantiles models and averages alternative calibration binning schemes rather than selecting a single fixed partition. It was proposed as a computationally tractable nonparametric Bayesian postprocessor for binary classifiers. ⁵⁶ A simpler bespoke alternative for small sports datasets is Bayesian Platt calibration, placing priors on a, b and integrating their posterior uncertainty rather than reporting only maximum-likelihood parameters.

My practical calibration ordering is:

Base model	First method to try	Alternatives	Reason
Logistic / dynamic BT	None initially; then Platt or Bayesian logistic correction	Beta	Already uses a probabilistic link, so diagnose before adding flexibility.
Elo/Glicko/ TrueSkill-derived probability	Platt or beta	Isotonic/BBQ	Rating scale/performance variance often needs only modest recalibration. ⁵⁷
XGBoost/ LightGBM/ CatBoost	Beta or Platt	Isotonic with abundant data	Trees often rank well but their raw probability shape should be validated separately. Calibration literature demonstrates substantial learner- dependent distortions. ⁵⁷
Deep Sets/GNN/ Transformer	Temperature	Platt, beta, isotonic	Temperature scaling is exceptionally low variance and preserves neural ranking. ⁴⁰
Very small calibration sample	Bayesian Platt / BBQ	Platt	Bayesian shrinkage/model averaging provides protection against fitting an unstable calibration curve. ⁵⁶

The critical rule is that the calibrator must be fitted on **predictions not used to fit the base model**. In time-dependent games, these should ideally be out-of-time predictions. Fitting an isotonic or Platt mapping to the base model's training predictions leaks optimism into the final probabilities; the calibration literature uses held-out predictions for precisely this reason. ⁵⁸

Evaluation metrics

Log loss should usually be the primary objective:

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

It strongly penalizes confident wrong forecasts and is a proper probability score. Proper-scoring-rule theory establishes logarithmic scoring as a principled way to evaluate predictive distributions. ⁵⁹

Brier score is

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2.$$

Brier's original work introduced the squared-error probability score for probabilistic forecasting. ⁶⁰ Compared with log loss, it is less violently affected by a single near-zero probability assigned to an event that occurs.

AUC measures discrimination:

$$\text{AUC} = P(s^+ > s^-).$$

It is useful for determining whether one team is generally ranked above another, but because any monotone transformation leaves ranking intact, it cannot tell you whether a 75% forecast actually wins roughly 75% of the time. This is why AUC should accompany rather than replace Brier/log loss and calibration diagnostics. Calibration studies explicitly distinguish ranking/classification performance from probability quality. ⁶¹

Accuracy,

$$\frac{1}{N} \sum_i \mathbf{1}[(p_i > 0.5) = y_i],$$

is useful as an easily communicated secondary metric but discards nearly all information about forecast confidence. This is an important limitation of much of the esports literature: GCN-WP and the player-champion-experience study, for example, headline accuracy, while PandaSkill supplements accuracy with ECE. ⁶²

Reliability diagrams bin forecasts and plot

$$\bar{p}_b \text{ against } \bar{y}_b.$$

The ideal curve follows $y = x$. Guo et al. use reliability analysis in modern-network calibration, and PandaSkill explicitly plots role-specific calibration curves and reports ECE. ⁶³

I would additionally fit the calibration regression

$$\text{logit } P(Y = 1) = a + b \text{ logit}(\hat{p}).$$

An ideal model has $a = 0, b = 1$. A slope $b < 1$ indicates probabilities that are generally too extreme; $b > 1$ indicates probabilities that are too compressed.

The production scorecard I recommend is therefore:

Category	Metric	Priority
Probability accuracy	Log loss	Primary
Probability accuracy	Brier score	Primary
Calibration	Reliability diagram + calibration slope/ intercept	Primary
Discrimination	AUC	Secondary
Operational interpretability	Accuracy at 0.5	Secondary
Calibration summary	ECE	Diagnostic, not a substitute for the reliability curve
Robustness	Log loss/Brier by season, patch, roster- change, league, favorite strength	Primary for deployment

Time-series cross-validation and backtesting

The evaluation should simulate exactly what production would have known.

For an expanding-window backtest,

$$\mathcal{D}_{\text{train}}^{(k)} = [t_0, t_k], \quad \mathcal{D}_{\text{test}}^{(k)} = (t_k, t_k + h].$$

For a sliding window,

$$\mathcal{D}_{\text{train}}^{(k)} = [t_k - W, t_k], \quad \mathcal{D}_{\text{test}}^{(k)} = (t_k, t_k + h].$$

Rolling-origin evaluation repeatedly advances the forecast origin and is a standard way of obtaining more informative out-of-sample forecast evaluations. ⁶⁴ Dawid's prequential perspective is even more directly relevant: predictions are judged sequentially in the order in which observations become available. ⁶⁵

Ordinary random cross-validation should not be the default for this problem. Research on time-series cross-validation shows that its validity depends on assumptions about the dependence structure; in a match-

forecasting pipeline, the stronger practical issue is that ratings, rolling features, roster states and patch states are explicitly path-dependent and must be reconstructed as they would have existed historically. 66

A rigorous backtest should operate as:

```
for each forecast date t:
    use only information timestamped before forecast cutoff t
    construct current player/rating states
    construct recent-form features from matches < t
    construct lineup and synergy features from co-play < t
    predict every match in the test horizon
    store raw probability
    after each result becomes available:
        update rating/feature state
```

Hyperparameters should be selected using earlier walk-forward folds, then the final model should be tested on an untouched later period. The calibrator should either use a separate historical calibration window or be constructed from stacked out-of-time predictions from the inner walk-forward folds.

PandaSkill's pre-match experiment is a good domain-specific example: it trains its logistic forecasting layer on one year and evaluates the subsequent month, shifts forward one month, and repeats, explicitly citing rating drift and game-meta changes as motivations for the rolling design. 50

The test set should also contain stress slices. In particular, report performance separately for established versus changed rosters, new players, long inactivity, inter-region/inter-league games, major rule/patch changes and high/low model-confidence matches. A global Brier score can otherwise hide precisely the cases in which a sophisticated lineup model is supposed to add value.

Evidence, datasets, implementations, and ranked recommendation

Reported empirical benchmarks

There is no credible single leaderboard across sports and esports because studies use different forecast cutoffs, populations, seasons, train/test schemes and metrics. The table below should therefore be read as evidence about architecture classes, **not** as an apples-to-apples tournament.

Work	Domain / data	Forecast model	Validation setup	Reported benchmark	What it tells us
GCN-WP, Bisberg & Ferrara	Professional League of Legends	Graph convolution over match/team graph and >30 features	Best GCN experiment: LPL training, LCK validation, LCS test on 2020; SCOPE rating uses historical initialization/validation procedure. ⁶⁷	Best GCN-Cheby configuration 61.9% accuracy ; SCOPE/Elo 59.7% ; best listed RF 57.8% . ⁸	Graph structure can add predictive information, but accuracy-only and split differences limit claims about probability quality.
PandaSkill	37,388 professional LoL games, 4,927 players , worldwide, Sept. 2019–Sept. 2024. ⁵³	Role-specific XGBoost performance model → OpenSkill/TrueSkill variants → pre-match logistic forecast	One-year training window → following month test, shifted monthly. ⁵⁰	PScore+OpenSkill: 65.56% overall accuracy, 1.53% ECE . PScore+Meta_OpenSkill: 65.12%, 0.89% ECE . PScore+Meta_FFA_OpenSkill: 70.07% inter-region accuracy, 3.27% inter-region ECE . ⁵⁰	Strong evidence for player-level Bayesian ratings, contextual regional ratings and explicit rolling evaluation.
LinNet	NBA lineup data, multiple seasons	Network embedding of five-player lineups	Out-of-sample evaluation on later games/seasons in the study. ⁶⁸	69% accuracy , compared with 56% adjusted-plus-minus baseline; reported validation curves also indicated useful probability calibration. ⁶⁸	Full-lineup identity and interaction can add signal beyond additive player values.

Work	Domain / data	Forecast model	Validation setup	Reported benchmark	What it tells us
Player-champion experience DNN	League of Legends ranked games	Deep neural network using player experience with selected champions	Forecast after all champion selections but before gameplay	75.1% accuracy. ¹⁸	Composition expertise is highly predictive, but post-draft results should not be compared with earlier roster-only forecasts.
PandaSkill player-performance component	Same professional LoL corpus	Role-specific monotonic XGBoost using end-game player statistics	Five-fold evaluation of the <i>performance-scoring component</i> , not the pre-match model	90.74% accuracy , mean 0.93% ECE ; authors emphasize that high accuracy is expected because these are end-game statistics. ⁵³	Useful method for learning the signal used to update future ratings; not itself a pre-match forecast benchmark.

The conspicuous literature gap is the scarcity of **out-of-time log-loss and Brier-score reporting** in esports. Several otherwise interesting papers focus on accuracy, whereas proper-scoring-rule theory indicates that probability quality requires probability-sensitive metrics. ⁶⁹ Future work should report at least log loss, Brier, AUC and reliability plots on the same chronological test windows.

High-value datasets

Dataset / source	Scope	Best use
Oracle's Elixir	Public professional League of Legends match/player/team CSV data spanning major leagues and many other competitions. ⁷⁰	Building team/player histories, draft/context features, ratings and reproducible professional LoL backtests.
PandaSkill released data	Five-year worldwide professional LoL research dataset; paper reports 37,388 games and 4,927 players, and the authors released source code and data. ⁷¹	Reproducing player-performance → rating pipelines and cross-region experiments.

Dataset / source	Scope	Best use
ESTA / awpy-based data	1,558 professional Counter-Strike game logs with millions of actions/frames and hundreds of thousands of trajectories, released to support esports analytics and win-prediction research. ⁷²	Building CS player form, map/style, lineup and granular event-derived features.
NBA play-by-play / lineup data as used by LinNet and plus-minus research	Repeated five-player lineups and possession/stint structure are the natural input to lineup networks and RAPM-style models. ⁷³	Best laboratory for lineup/substitution and synergy methodology transferable to other 5v5 sports.

For a serious model, event data should generally be transformed into **historical player/team state** rather than fed into a pre-match model as a giant collection of raw past-game statistics. That distinction makes the feature store causal, compact and transferable across roster moves.

Best primary works to read

Work	Why it matters
Király & Qian — “Modelling Competitive Sports: Bradley-Terry-Élő Models...”	Best bridge between Elo, Bradley-Terry, online learning, covariates and supervised prediction. ²⁹
Glickman — “Parameter Estimation in Large Dynamic Paired Comparison Experiments”	Statistical foundation for dynamic paired comparisons and uncertainty-aware ratings. ²⁴
Herbrich, Minka & Graepel — “TrueSkill: A Bayesian Skill Rating System”	Canonical player-level Bayesian rating model for teams/multiple players. ¹⁹
TrueSkill Through Time	Essential extension when latent skill evolves and historical trajectory estimation matters. ³
Zaheer et al. — “Deep Sets”	Theoretical architecture for permutation-invariant team/member aggregation. ³⁸
Kipf & Welling — “Semi-Supervised Classification with Graph Convolutional Networks”	Foundational GCN architecture for relational player/team modeling. ⁴¹
Bisberg & Ferrara — GCN-WP	Direct application of graph convolution to professional esports pre-match prediction. ⁶⁷
Pelechrinis — LinNet	Strong example of lineup-level embeddings and prediction rather than individual-only player ratings. ⁶⁸
Hypergraph Adjusted Plus-Minus	One of the most directly relevant formulations of higher-order lineup synergy. ⁴³

Work	Why it matters
PandaSkill	Probably the most useful recent esports reference for combining role-aware ML, Bayesian ratings, regional context and rolling pre-match evaluation. ⁷⁴
Guo et al. — “On Calibration of Modern Neural Networks”	Essential calibration reference; temperature scaling. ⁴⁰
Kull, Silva Filho & Flach — Beta Calibration	Particularly valuable post-hoc calibration method for binary probabilities. ⁵⁵
Naeini, Cooper & Hauskrecht — Bayesian Binning into Quantiles	Useful Bayesian calibration alternative. ⁵⁶
Tashman — rolling-origin forecasting evaluation	Core methodological reference for honest chronological forecast experiments. ⁶⁴
Dawid — prequential approach	Fundamental conceptual framework for evaluating forecasts sequentially as data arrive. ⁶⁵

Open-source implementations

Implementation	What to use it for	Assessment
sublee/trueskill	Python TrueSkill experiments and player/team rating prototypes. ⁷⁵	Excellent reference implementation for baseline research.
OpenSkill / openskill.py	Open multiplayer/team Bayesian ratings based on Weng-Lin-style models. ²⁶	My preferred practical starting point for a custom open 5v5 player-rating component.
sublee/glicko2	Glicko-2 baseline in Python. ⁷⁶	Useful baseline where uncertainty/volatility matter but full TrueSkill machinery is unnecessary.
TrueSkill Through Time implementations	Dynamic skill trajectory modeling; JSS work documents Python/Julia/R software. ⁷⁷	Best open research basis for temporally smoothed Bayesian player skill.
PandaScore/ PandaSkill	Full code/data corresponding to the PandaSkill LoL paper. ⁷⁸	Highest-value domain-specific reference implementation in this survey.
GCN-WP code / GCN base implementation	Graph-based esports model experimentation; the GCN-WP paper identifies its implementation lineage, and Kipf’s GCN implementation is publicly available. ⁷⁹	Useful research prototype, though I would modernize the architecture for production.

Implementation	What to use it for	Assessment
XGBoost	Hybrid tabular match model over ratings, context and lineup features. ⁸⁰	Top general-purpose base learner recommendation.
LightGBM	Faster/memory-efficient GBDT alternative for larger feature stores. ⁸¹	Equally strong candidate; benchmark it against XGBoost rather than assuming one wins.
Deep Sets reference implementation	Permutation-invariant five-player encoders. The NeurIPS entry links the authors' implementation. ⁸²	Strong starting point for a neural roster encoder.
Kipf GCN	Canonical graph-convolution reference code. ⁸³	Useful conceptual implementation; modern PyTorch Geometric/DGL-style rewrites are more convenient operationally.

Final ranked architecture recommendation

The first-ranked production design is a player-level Bayesian/dynamic rating engine feeding a gradient-boosted matchup model. Let each player carry

$$r_{i,t} = (\mu_{i,t}, \sigma_{i,t}, \text{role, recent form, context states}),$$

preferably using OpenSkill/TrueSkill-like updates or a custom state-space Bradley-Terry model. This gives roster transferability, uncertainty and a strong low-dimensional prior. TrueSkill/OpenSkill and PandaSkill provide direct precedents for this decomposition. ⁸⁴

Construct the match vector from **differences** wherever symmetry permits:

$$x_t = [R_A - R_B, U_A - U_B, F_A - F_B, S_A - S_B, C_A - C_B, \text{context}].$$

Use XGBoost or LightGBM to estimate nonlinear residual matchups,

$$z_t = F_{\text{GBDT}}(x_t), \quad p_t = \sigma(z_t),$$

then fit beta/Platt/isotonic calibration on strictly later held-out predictions. XGBoost and LightGBM provide highly developed implementations of boosted decision trees; calibration research supplies the probability correction layer. ⁸⁵

Add synergy gradually:

$$S_A = \sum_{i < j} \gamma_{ij}$$

first, then low-rank pair embeddings, then full-lineup/hypergraph or GNN representations only after backtests demonstrate enough support for the added degrees of freedom. LinNet and HAPM make a strong

case that lineup interactions contain information, but they also illustrate why sparse lineup exposure is the core statistical difficulty. ³¹

The **second-ranked architecture**, and the one I would use as the most rigorous benchmark, is a fully hierarchical dynamic Bradley–Terry model:

$$\text{logit } P(A > B) = \sum_{i \in A} \theta_{i,t} - \sum_{j \in B} \theta_{j,t} + \beta^\top x_t + S_A - S_B$$

with

$$\theta_{i,t} \sim N(\theta_{i,t-1}, q_i \Delta t), \quad \gamma_{ij} \sim N(0, \tau_\gamma^2).$$

It is harder to engineer but gives a coherent answer to almost every problem in this domain: ratings, uncertainty, player movement, role priors, temporal dynamics, competition effects and regularized synergy. Dynamic paired-comparison models, TrueSkill and adjusted-plus-minus research provide the main pieces of this construction. ⁸⁶

The **third-ranked architecture** is a Deep Sets or graph model that consumes these dynamic player states:

$$\begin{aligned} h_i &= \phi(r_{i,t}, \text{form}_i, \text{role}_i), \\ z_A &= \text{AttentionPool}(h_1, \dots, h_5), \\ p &= \sigma(g(z_A, z_B, z_A \odot z_B, \text{context})). \end{aligned}$$

This is a more statistically defensible neural architecture than simply putting five player IDs and hundreds of features into an MLP because it explicitly respects team composition structure. Deep Sets supplies permutation invariance, GNNs supply relational learning, and GCN-WP demonstrates the viability of graph-based esports prediction. ¹³

The **standalone rating systems remain essential baselines**. Every experiment should include at least team Elo, an uncertainty-aware rating such as Glicko, and a player-aware Bayesian system such as OpenSkill/TrueSkill. Elo's continued theoretical and empirical competitiveness means that a complex model that cannot consistently beat a causally implemented Elo baseline on future log loss or Brier score has probably learned complexity rather than useful signal. ¹

The resulting recommended research stack is therefore:

Baseline:	Elo / Bradley–Terry
Player state:	OpenSkill / TrueSkill / dynamic BT
Temporal layer:	state-space dynamics + EWMA features
Roster layer:	current five + uncertainty over substitutions
Synergy layer:	regularized pairs → embeddings/hypergraph
Predictor:	XGBoost/LightGBM first
Advanced predictor:	Deep Sets / GNN with rating priors
Calibration:	beta/Platt for trees; temperature for neural nets
Selection metric:	out-of-time log loss + Brier
Diagnostics:	AUC + reliability + calibration slope
Validation:	walk-forward / rolling-origin prequential backtest.

That stack combines the most durable idea from classical rating systems—a **compact, continuously updated latent representation of ability**—with the principal advantage of modern machine learning—**nonlinear modeling of context and interaction**—while retaining the forecasting discipline that many published game-prediction studies still lack: strict historical information cutoffs, chronological validation, proper scoring rules and explicit probability calibration. ⁸⁷

¹ ¹¹ ¹⁷ ²³ ²⁹ ⁸⁷ Modelling Competitive Sports: Bradley-Terry-Élő Models for Supervised and On-Line Learning of Paired Competition Outcomes

https://arxiv.org/abs/1701.08055?utm_source=chatgpt.com

² ¹² ¹⁵ ²⁰ ²⁴ ²⁷ ³² ⁸⁶ Parameter Estimation in Large Dynamic Paired Comparison ...

https://academic.oup.com/jrsssc/article/48/3/377/6990661?utm_source=chatgpt.com

³ TrueSkill Through Time: Revisiting the History of Chess

https://www.microsoft.com/en-us/research/publication/trueskill-through-time-revisiting-the-history-of-chess/?utm_source=chatgpt.com

⁴ ²¹ ³¹ ⁶⁸ ⁷³ <https://arxiv.org/abs/1707.01855>

<https://arxiv.org/abs/1707.01855>

⁵ ⁸⁰ ⁸⁵ <https://arxiv.org/abs/1603.02754>

<https://arxiv.org/abs/1603.02754>

⁶ ³⁴ ³⁹ ⁴⁸ ⁵⁰ ⁵² ⁵³ ⁷¹ <https://arxiv.org/html/2501.10049v2>

<https://arxiv.org/html/2501.10049v2>

⁷ ¹³ ³⁸ ⁸² <https://papers.nips.cc/paper/6931-deep-sets>

<https://papers.nips.cc/paper/6931-deep-sets>

⁸ ⁴² ⁶² ⁶⁷ ⁶⁹ ⁷⁹ <https://arxiv.org/html/2207.13191v1>

<https://arxiv.org/html/2207.13191v1>

⁹ ⁶⁰ [https://journals.ametsoc.org/view/journals/mwre/](https://journals.ametsoc.org/view/journals/mwre/78/1/1520-0493_1950_078_0001_vofeit_2_0_co_2.xml)

[78/1/1520-0493_1950_078_0001_vofeit_2_0_co_2.xml](https://journals.ametsoc.org/view/journals/mwre/78/1/1520-0493_1950_078_0001_vofeit_2_0_co_2.xml)

https://journals.ametsoc.org/view/journals/mwre/78/1/1520-0493_1950_078_0001_vofeit_2_0_co_2.xml

10 40 58 63 <https://proceedings.mlr.press/v70/guo17a.html>

<https://proceedings.mlr.press/v70/guo17a.html>

14 19 44 84 **TrueSkill™: A Bayesian Skill Rating System**

https://proceedings.neurips.cc/paper/2006/hash/f44ee263952e65b3610b8ba51229d1f9-Abstract.html?utm_source=chatgpt.com

16 36 64 <https://www.sciencedirect.com/science/article/abs/pii/S0169207000000650>

<https://www.sciencedirect.com/science/article/abs/pii/S0169207000000650>

18 <https://arxiv.org/abs/2108.02799>

<https://arxiv.org/abs/2108.02799>

22 45 **A Spectral Approach for the Dynamic Bradley-Terry Model**

https://arxiv.org/abs/2307.16642?utm_source=chatgpt.com

25 **Mark Glickman's Research Page**

https://www.glicko.net/research.html?utm_source=chatgpt.com

26 <https://github.com/vivekjoshi/openskill.py>

<https://github.com/vivekjoshi/openskill.py>

28 **A Practical Guide to the Multi-Dimensional Elo Rating System**

https://www.preprints.org/manuscript/202606.0161?utm_source=chatgpt.com

30 **microsoft.com**

https://www.microsoft.com/en-us/research/wp-content/uploads/2007/01/NIPS2006_0688.pdf

33 74 <https://arxiv.org/abs/2501.10049>

<https://arxiv.org/abs/2501.10049>

35 54 <https://dl.acm.org/doi/10.1145/775047.775151>

<https://dl.acm.org/doi/10.1145/775047.775151>

37 59 <https://stat.uw.edu/research/tech-reports/strictly-proper-scoring-rules-prediction-and-estimation>

<https://stat.uw.edu/research/tech-reports/strictly-proper-scoring-rules-prediction-and-estimation>

41 <https://arxiv.org/abs/1609.02907>

<https://arxiv.org/abs/1609.02907>

43 51 <https://arxiv.org/html/2403.20214v3>

<https://arxiv.org/html/2403.20214v3>

46 49 <https://arxiv.org/html/2007.10550v1>

<https://arxiv.org/html/2007.10550v1>

47 **NBA2Vec: Dense feature representations of NBA players**

https://arxiv.org/abs/2302.13386?utm_source=chatgpt.com

55 <https://proceedings.mlr.press/v54/kull17a.html>

<https://proceedings.mlr.press/v54/kull17a.html>

56 <https://ojs.aaai.org/index.php/AAAI/article/view/9602>

<https://ojs.aaai.org/index.php/AAAI/article/view/9602>

57 61 <https://dl.acm.org/doi/10.1145/1102351.1102430>

<https://dl.acm.org/doi/10.1145/1102351.1102430>

65 <https://academic.oup.com/jrsssa/article/147/2/278/7106293>

<https://academic.oup.com/jrsssa/article/147/2/278/7106293>

66 <https://www.sciencedirect.com/science/article/abs/pii/S0167947317302384>

<https://www.sciencedirect.com/science/article/abs/pii/S0167947317302384>

70 <https://oracleselixir.com/>

<https://oracleselixir.com/>

72 <https://arxiv.org/abs/2209.09861>

<https://arxiv.org/abs/2209.09861>

75 <https://github.com/sublee/trueskill>

<https://github.com/sublee/trueskill>

76 <https://github.com/sublee/glicko2>

<https://github.com/sublee/glicko2>

77 <https://github.com/glandfried/TrueSkillThroughTime.py>

<https://github.com/glandfried/TrueSkillThroughTime.py>

78 <https://github.com/PandaScore/PandaSkill>

<https://github.com/PandaScore/PandaSkill>

81 <https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>

<https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>

83 <https://github.com/tkipf/gcn>

<https://github.com/tkipf/gcn>